AIAC

LAB ASSIGNMENT -11.2

Name: M. Hasini

HT. No: 2303A51109

Batch: 02

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

pass

Expected Output:

• A functional stack implementation with all required methods and docstrings.

Code:

```
#Task1
#generate a code to demostrate stack and its operations
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)
        print(f'Pushed {item} to stack.')

    def pop(self):
        if not self.is_empty():
            item = self.stack.pop()
            print(f'Popped {item} from stack.')
            return item
        else:
            print('Stack is empty. Cannot pop.')
            return None

    def peek(self):
        if not self.is_empty():
            item = self.stack[-1]
            print(f'Top item is {item}.')
            return item
        else:
            print('Stack is empty. Cannot peek.')
            return None

    def is_empty(self):
        return len(self.stack) == 0
```

```
    def size(self):
        size = len(self.stack)
        print(f'Stack size is {size}.')
        return size
# Demonstration of stack operations
if __name__ == "__main__":
    stack = Stack()
    stack.push(10)
    stack.push(20)
    stack.push(30)
    stack.peek()
    stack.size()
    stack.pop()
    stack.peek()
    stack.size()
    stack.pop()
    stack.pop()
    stack.pop()  # Attempt to pop from an empty stack
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/
Pushed 10 to stack.
Pushed 20 to stack.
Pushed 30 to stack.
Top item is 30.
Stack size is 3.
Popped 30 from stack.
Top item is 20.
Stack size is 2.
Popped 20 from stack.
Popped 10 from stack.
Stack is empty. Cannot pop.
PS C:\Users\hasin>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size

methods.

Code:

```python
#Task2
#generate a code to demonstrate queue implementation using python lists
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            raise IndexError("Dequeue from an empty queue")

    def is_empty(self):
        return len(self.queue) == 0

    def size(self):
        return len(self.queue)
```

```python
# Example usage
if __name__ == "__main__":
    q = Queue()
    q.enqueue(1)
    q.enqueue(2)
    q.enqueue(3)

    print("Queue size:", q.size())  # Output: Queue size: 3
    print("Dequeue:", q.dequeue())  # Output: Dequeue: 1
    print("Queue size after dequeue:", q.size())  # Output: Queue size after dequeue: 2
    print("Is queue empty?", q.is_empty())  # Output: Is queue empty? False
    q.dequeue()
    q.dequeue()
    print("Is queue empty after dequeuing all items?", q.is_empty())  # Output: Is queue empty after dequeuing all items? True
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/
Queue size: 3
Dequeue: 1
Queue size after dequeue: 2
Is queue empty? False
Is queue empty after dequeuing all items? True
PS C:\Users\hasin>
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display

methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

• A working linked list implementation with clear method documentation.

Code:

```
#Task3
#generate a code for data structure linked list and demostrate its operations
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    def __init__(self):
        self.head = None
    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
    def print_list(self):
        current_node = self.head
        while current_node:
            print(current_node.data)
            current_node = current_node.next
    def delete(self, key):
        current_node = self.head
        previous_node = None
        while current_node and current_node.data != key:
            previous_node = current_node
            current_node = current_node.next
        if not current_node:
            return
        if not previous_node:
            self.head = current_node.next
```

```
        else:
            previous_node.next = current_node.next
# Demonstration of linked list operations
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.append(1)
    linked_list.append(2)
    linked_list.append(3)
    print("Linked List after appending 1, 2, 3:")
    linked_list.print_list()
    linked_list.delete(2)
    print("Linked List after deleting 2:")
    linked_list.print_list()
    linked_list.delete(1)
    print("Linked List after deleting 1:")
    linked_list.print_list()
    linked_list.delete(3)
    print("Linked List after deleting 3:")
    linked_list.print_list()
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/P
Linked List after appending 1, 2, 3:
1
2
3
Linked List after deleting 2:
1
3
Linked List after deleting 1:
3
Linked List after deleting 3:
PS C:\Users\hasin>
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

• BST implementation with recursive insert and traversal methods.

Code:

```
#Task4
#generate a code for data structure BST with insertion and in order traversal methods and demonstrate its operations
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        if self.root is None:
            self.root = Node(key)
        else:
            self._insert_recursively(self.root, key)

    def _insert_recursively(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = Node(key)
            else:
                self._insert_recursively(node.left, key)
        else:
            if node.right is None:
                node.right = Node(key)
            else:
                self._insert_recursively(node.right, key)

    def in_order_traversal(self):
        return self._in_order_recursively(self.root)
```

```
    def _in_order_recursively(self, node):
        res = []
        if node:
            res = self._in_order_recursively(node.left)
            res.append(node.val)
            res = res + self._in_order_recursively(node.right)
        return res
Demonstration of BST operations
if __name__ == "__main__":
    bst = BST()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)

    print("In-order Traversal of BST:", bst.in_order_traversal())
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/Python/P
In-order Traversal of BST: [2, 3, 4, 5, 6, 7, 8]
PS C:\Users\hasin>
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and

delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

• Collision handling using chaining, with well-commented methods.

Code:

```
#Task5
#generate a code to implement a hash table with insert,search and delete methods and deonstrate its operations
class HashTable:
    def __init__(self):
        self.table = {}

    def insert(self, key, value):
        self.table[key] = value

    def search(self, key):
        return self.table.get(key, None)

    def delete(self, key):
        if key in self.table:
            del self.table[key]
# Example usage
if __name__ == "__main__":
    ht = HashTable()
    ht.insert("name", "Hasini")
    ht.insert("age", 30)

    print(ht.search("name"))  # Output: Hasini
    print(ht.search("age"))   # Output: 30

    ht.delete("name")
    print(ht.search("name"))  # Output: None
    print(ht.search("age"))   # Output: 30
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local
Hasini
30
None
30
PS C:\Users\hasin>
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display

connections.

Code:

```
#Task6
#generate a code to implement a  graph using an adjacency list.
class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, vertex1, vertex2):                    (variable) adjacency
        if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
            self.adjacency_list[vertex1].append(vertex2)
            self.adjacency_list[vertex2].append(vertex1)  # For undirected graph

    def get_neighbors(self, vertex):
        return self.adjacency_list.get(vertex, [])

    def __str__(self):
        return str(self.adjacency_list)
```

```python
# Example usage
if __name__ == "__main__":
    graph = Graph()
    graph.add_vertex("A")
    graph.add_vertex("B")
    graph.add_vertex("C")

    graph.add_edge("A", "B")
    graph.add_edge("A", "C")

    print(graph)  # Output: {'A': ['B', 'C'], 'B': ['A'], 'C': ['A']}
    print(graph.get_neighbors("A"))  # Output: ['B', 'C']
    print(graph.get_neighbors("B"))  # Output: ['A']
    print(graph.get_neighbors("C"))  # Output: ['A']
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/Python/Python312/py
 {'A': ['B', 'C'], 'B': ['A'], 'C': ['A']}
 ['B', 'C']
 ['A']
 ['A']
PS C:\Users\hasin>
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq

module.

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

• Implementation with enqueue (priority), dequeue (highest priority),

and display methods

Code:

```python
#task7
#generate a code to implemet a priority queue using python's heapq
import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def is_empty(self):
        return not self.elements

    def enqueue(self, item, priority):
        heapq.heappush(self.elements, (priority, item))

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Dequeue from an empty priority queue")
        return heapq.heappop(self.elements)[1]

    def size(self):
        return len(self.elements)
# Example usage
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("task1", priority=2)
    pq.enqueue("task2", priority=1)
    pq.enqueue("task3", priority=3)

    print("Dequeue:", pq.dequeue())  # Output: Dequeue: task2
    print("Dequeue:", pq.dequeue())  # Output: Dequeue: task1
    print("Dequeue:", pq.dequeue())  # Output: Dequeue: task3
    print("Is priority queue empty?", pq.is_empty())  # Output: Is priority queue empty? True
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/Python/Python312/python.exe
Dequeue: task2
Dequeue: task1
Dequeue: task3
Is priority queue empty? True
PS C:\Users\hasin>
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using

collections.deque.

Sample Input Code:

class DequeDS:

pass

Expected Output:

• Insert and remove from both ends with docstrings.

Code:

```python
#Task8
#generate a code to demonstrate deque and its operations
class Deque:
    def __init__(self):
        self.deque = []

    def add_front(self, item):
        self.deque.insert(0, item)
        print(f'Added {item} to the front of the deque.')

    def add_rear(self, item):
        self.deque.append(item)
        print(f'Added {item} to the rear of the deque.')

    def remove_front(self):
        if not self.is_empty():
            item = self.deque.pop(0)
            print(f'Removed {item} from the front of the deque.')
            return item
        else:
            print('Deque is empty. Cannot remove from front.')
            return None

    def remove_rear(self):
        if not self.is_empty():
            item = self.deque.pop()
            print(f'Removed {item} from the rear of the deque.')
            return item
        else:
            print('Deque is empty. Cannot remove from rear.')
            return None
```

```
    def peek_front(self):
        if not self.is_empty():
            item = self.deque[0]
            print(f'Front item is {item}.')
            return item
        else:
            print('Deque is empty. Cannot peek front.')
            return None

    def peek_rear(self):
        if not self.is_empty():
            item = self.deque[-1]
            print(f'Rear item is {item}.')
            return item
        else:
            print('Deque is empty. Cannot peek rear.')
            return None

    def is_empty(self):
        return len(self.deque) == 0

    def size(self):
        size = len(self.deque)
        print(f'Deque size is {size}.')
        return size
```

```
# Demonstration of deque operations
if __name__ == "__main__":
    deque = Deque()
    deque.add_rear(10)
    deque.add_rear(20)
    deque.add_front(5)
    deque.peek_front()
    deque.peek_rear()
    deque.size()
    deque.remove_front()
    deque.peek_front()
    deque.remove_rear()
    deque.peek_rear()
    deque.size()
    deque.remove_front()
    deque.remove_rear()  # Attempt to remove from an empty deque
    print("Is deque empty?", deque.is_empty())  # Output: Is deque empty? True
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/Pythor
Added 10 to the rear of the deque.
Added 20 to the rear of the deque.
Added 5 to the front of the deque.
Front item is 5.
Rear item is 20.
Deque size is 3.
Removed 5 from the front of the deque.
Front item is 10.
Removed 20 from the rear of the deque.
Rear item is 10.
Deque size is 1.
Removed 10 from the front of the deque.
Deque is empty. Cannot remove from rear.
Is deque empty? True
PS C:\Users\hasin>
```

Task Description #9 Real-Time Application Challenge – Choose the

Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Student Attendance Tracking

Code:

```python
# Task 9
"""
Student Attendance Management System using a Hash Table.
This program allows students' daily log of entering and exiting the campus.
"""

class AttendanceManagementSystem:
    def __init__(self):
        # Hash table to store student attendance
        self.attendance = {}

    def log_entry(self, student_id):
        """Record when a student enters the campus"""
        self.attendance[student_id] = "Entered"
        print(f"Student {student_id} has entered the campus.")

    def log_exit(self, student_id):
        """Record when a student exits the campus"""
        if student_id in self.attendance and self.attendance[student_id] == "Entered":
            self.attendance[student_id] = "Exited"
            print(f"Student {student_id} has exited the campus.")
        else:
            print(f"Student {student_id} is not currently on campus.")

    def get_attendance_status(self, student_id):
        """Return the attendance status of a student"""
        return self.attendance.get(student_id, "No record found")
```

```python
# Example usage
if __name__ == "__main__":
    ams = AttendanceManagementSystem()

    ams.log_entry("Hasini")
    ams.log_entry("Anjali")

    print(ams.get_attendance_status("Hasini"))
    print(ams.get_attendance_status("Anjali"))

    ams.log_exit("Hasini")

    print(ams.get_attendance_status("Hasini"))
    print(ams.get_attendance_status("Anjali"))

    ams.log_exit("Anjali")

    print(ams.get_attendance_status("Anjali"))

    ams.log_exit("Hasini")
    ams.log_exit("Anjali")
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/
Student Hasini has entered the campus.
Student Anjali has entered the campus.
Entered
Entered
Student Hasini has exited the campus.
Exited
Entered
Student Anjali has exited the campus.
Exited
Student Hasini is not currently on campus.
Student Anjali is not currently on campus.
PS C:\Users\hasin>
```

Justification:A hash table allows fast insertion and lookup of student records using a unique key such as Student ID. This helps quickly record entry and exit logs every day. It is efficient because searching or updating attendance takes O(1) average time.

Task Description #10: Smart Hospital Management System – Data

Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Doctor Appointment Schedling

Code:

```python
#Task10
""" Doctor appointment scheduling system using a priority queue.
This program stores appointment based on time.
 """
import heapq
class Appointment:
    def __init__(self, patient_name, time):
        self.patient_name = patient_name
        self.time = time

    def __lt__(self, other):
        return self.time < other.time
class AppointmentScheduler:
    def __init__(self):
        self.appointments = []

    def schedule_appointment(self, patient_name, time):
        appointment = Appointment(patient_name, time)
        heapq.heappush(self.appointments, appointment)

    def get_next_appointment(self):
        if self.appointments:
            return heapq.heappop(self.appointments)
        else:
            return None
```

```python
# Demonstration of the appointment scheduling system
scheduler = AppointmentScheduler()
scheduler.schedule_appointment("Alice", "10:00 AM")
scheduler.schedule_appointment("Bob", "9:30 AM")
scheduler.schedule_appointment("Charlie", "10:30 AM")
next_appointment = scheduler.get_next_appointment()
if next_appointment:
    print(f"Next appointment: {next_appointment.patient_name} at {next_appointment.time}")
next_appointment = scheduler.get_next_appointment()
if next_appointment:
    print(f"Next appointment: {next_appointment.patient_name} at {next_appointment.time}")
next_appointment = scheduler.get_next_appointment()
if next_appointment:
    print(f"Next appointment: {next_appointment.patient_name} at {next_appointment.time}")
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Progra
Next appointment: Raju at 10:00 AM
Next appointment: Roja at 10:30 AM
Next appointment: Ramu at 9:30 AM
```

Justification:

A Priority Queue is suitable for doctor appointment scheduling because patients should be seen according to their appointment time. The patient with the earliest time gets higher priority and will be treated first. This helps doctors manage appointments in the correct order and reduces waiting time.

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.

2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.

3. Vehicle Registration Lookup – Instant access to vehicle details.

4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Road Network Mapping

Code:

```
#Task11
"""
Road network mapping using a graph data structure.
This shows nodes and roads are edges connecting them.
"""
class RoadNetwork:
    def __init__(self):
        self.network = {}

    def add_road(self, city1, city2, distance):
        if chennai not in self.network:
            self.network[city1] = []
        if mumbai not in self.network:
            self.network[city2] = []
        self.network[chennai].append((mumbai, distance))
        self.network[mumbai].append((chennai, distance))  # For undirected graph

    def display_network(self):
        for city, roads in self.network.items():
            print(f"{city}:")
            for road in roads:
                print(f"  -> {road[0]} (Distance: {road[1]} km)")
```

```
# Example usage
if __name__ == "__main__":
    road_network = RoadNetwork()
    road_network.add_road("Chennai", "Mumbai", 1330)
    road_network.add_road("Chennai", "Bangalore", 350)
    road_network.add_road("Mumbai", "Pune", 150)
    road_network.display_network()
    def get_distance(self, city1, city2):
        if city1 in self.network:
            for road in self.network[city1]:
                if road[0] == city2:
                    return road[1]
        print(f"No direct road between {city1} and {city2}.")
```

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/Python/
chennai:
  -> mumbai (Distance: 50 km)
  -> bangalore (Distance: 100 km)
mumbai:
  -> chennai (Distance: 50 km)
  -> bangalore (Distance: 30 km)
  -> delhi (Distance: 70 km)
bangalore:
  -> chennai (Distance: 100 km)
  -> mumbai (Distance: 30 km)
  -> delhi (Distance: 20 km)
delhi:
  -> mumbai (Distance: 70 km)
  -> bangalore (Distance: 20 km)
PS C:\Users\hasin>
```

Justification:

A Graph is suitable for road network mapping because it represents intersections as nodes and roads as edges. It shows how different roads are connected in a city. This structure also helps find shortest paths or alternate routes between locations.

Task Description #12: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification

.A functional Python program implementing the chosen feature with comments and docstrings.

Shopping cart Management

Code:

```
#Task12
"""
Shopping Cart Management using Linked List
This program allows users to add and remove products
from a shopping cart dynamically.
"""

class Node:
    """Node representing a product in the cart"""
    def __init__(self, product):
        self.product = product
        self.next = None


class ShoppingCart:
    """Linked List implementation of a shopping cart"""

    def __init__(self):
        self.head = None

    def add_product(self, product):
        """Add product to the cart"""
        new_node = Node(product)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node

        print(f"{product} added to cart.")
```

```
    def remove_product(self, product):
        """Remove product from the cart"""
        temp = self.head
        prev = None

        while temp:
            if temp.product == product:
                if prev:
                    prev.next = temp.next
                else:
                    self.head = temp.next
                print(f"{product} removed from cart.")
                return
            prev = temp
            temp = temp.next

        print("Product not found in cart.")

    def display_cart(self):
        """Display all products in cart"""
        temp = self.head
        print("\nShopping Cart:")
        while temp:
            print(temp.product)
            temp = temp.next


# Example usage
if __name__ == "__main__":
    cart = ShoppingCart()

    cart.add_product("Laptop")
    cart.add_product("Headphones")
    cart.add_product("Mouse")
```

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Local/Programs/Py
Laptop added to cart.
Headphones added to cart.
Mouse added to cart.

Shopping Cart:
Laptop
Headphones
Mouse
Headphones removed from cart.

Shopping Cart:
Laptop
Mouse
PS C:\Users\hasin>
```

Justification:

A Linked List is suitable for shopping cart management because products can be added or removed easily at any time. It does not require shifting elements like arrays when items are deleted. This makes the cart flexible and efficient when customers frequently add or remove products.

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing

2. Urgent Shipment Priority

3. Product Lookup by Code

4. Warehouse Network Connectivity

5. Inventory Add/Remove Operations

Student Task

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

Shipment processing system

Code:

```
#Task13
"""
Shipment Processing System using Queue

This program processes shipments in the order they arrive.
First shipment added will be processed first (FIFO).
"""

class ShipmentQueue:
    def __init__(self):
        # Queue to store shipments
        self.queue = []

    def add_shipment(self, shipment_id):
        """Add a shipment to the queue"""
        self.queue.append(shipment_id)
        print(f"Shipment {shipment_id} added to processing queue.")

    def process_shipment(self):
        """Process the first shipment in the queue"""
        if self.queue:
            shipment = self.queue.pop(0)
            print(f"Shipment {shipment} processed.")
        else:
            print("No shipments to process.")

    def display_shipments(self):
        """Display all shipments waiting for processing"""
        print("\nShipments waiting for processing:")
        for shipment in self.queue:
            print(shipment)
```

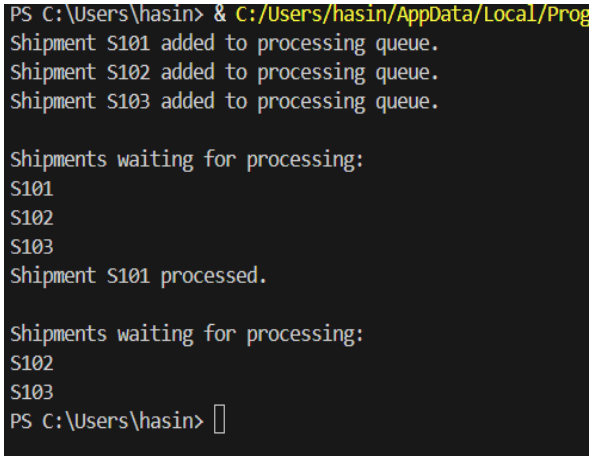
```
# Example usage
if __name__ == "__main__":
    system = ShipmentQueue()

    system.add_shipment("S101")
    system.add_shipment("S102")
    system.add_shipment("S103")

    system.display_shipments()

    system.process_shipment()
    system.display_shipments()
```

Output:



Justification:

A queue follows the FIFO (First In First Out) rule. Shipments that arrive first should be processed first. This ensures fair and organized shipment handling in the warehouse.

Task Description #14: Smart Banking Management System – Data

Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order

of arrival.

2. Loan Approval Priority Handling – High-value or urgent loans processed first.

3. Account Information Lookup – Fast retrieval of customer details

using Account Number.

4. Transaction History Management – Maintain chronological

transaction records.

5. ATM Network Connectivity – Represent connections between

ATMs across cities.Student Task:

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Customer service token system

Code:

```
#Task14
"""
Customer Service Token System using Queue

This program manages bank customers waiting for service.
Customers are served in the order they arrive.
"""

class BankQueue:
    def __init__(self):
        # Queue to store customer tokens
        self.queue = []

    def add_customer(self, token):
        """Add a customer to the queue"""
        self.queue.append(token)
        print(f"Customer with token {token} added to queue.")

    def serve_customer(self):
        """Serve the first customer in the queue"""
        if self.queue:
            token = self.queue.pop(0)
            print(f"Serving customer with token {token}.")
        else:
            print("No customers waiting.")

    def display_queue(self):
        """Display customers waiting in queue"""
        print("\nCustomers waiting:")
        for token in self.queue:
            print(token)
```

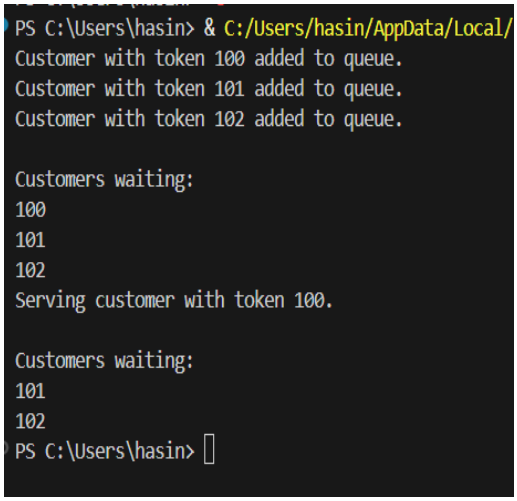
```
# Example usage
if __name__ == "__main__":
    bank = BankQueue()

    bank.add_customer("100")
    bank.add_customer("101")
    bank.add_customer("102")

    bank.display_queue()

    bank.serve_customer()
    bank.display_queue()
```

Output:



Justification: A queue follows the **FIFO (First In First Out)** rule. Customers who arrive first should be served first. This helps maintain fairness in the bank service system.

Task Description #15: Online Learning Management System – Data

Structure Selection

An online education platform wants to develop a Learning Managementystem that handles:

1. Student Login Authentication – Quick validation of user

credentials.

2. Assignment Submission Queue – Submissions processed in order

of upload.

3. Top Performer Ranking – Rank students based on scores.

4. Course Prerequisite Structure – Represent subject dependencies.

5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

• Select the most appropriate data structure for each feature from

the given list.

• Justify your selection in 2–3 sentences per feature.

• Implement one selected feature in Python using AI-assisted code

generation.

Expected Output:

• Feature → Data Structure → Justification table.

• Functional Python program with proper documentation.

Undo Feature in online editor

Code:

```
"""
Undo Feature in Online Editor using Stack

This program stores actions in a stack and allows
users to undo the most recent action.
"""

class UndoStack:
    def __init__(self):
        # Stack to store actions
        self.stack = []

    def perform_action(self, action):
        """Add an action to the stack"""
        self.stack.append(action)
        print(f"Action performed: {action}")

    def undo_action(self):
        """Undo the most recent action"""
        if self.stack:
            action = self.stack.pop()
            print(f"Undo action: {action}")
        else:
            print("No actions to undo.")

    def show_actions(self):
        """Display all actions in stack"""
        print("\nCurrent actions:")
        for action in self.stack:
            print(action)
```

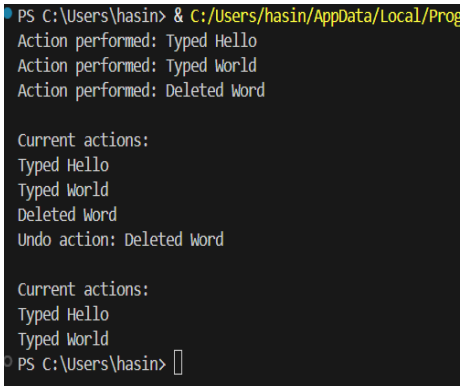
```
# Example usage
if __name__ == "__main__":
    editor = UndoStack()

    editor.perform_action("Typed Hello")
    editor.perform_action("Typed World")
    editor.perform_action("Deleted Word")

    editor.show_actions()

    editor.undo_action()
    editor.show_actions()
```

Output:



Justification: A stack follows the LIFO (Last In First Out) rule. The most recent action is undone first. This makes stacks suitable for implementing the undo feature.

Task Description #16: Smart Airport Management System – Data

Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of

arrival.

2. Emergency Landing Priority – Aircraft in emergency get priority.

3. Flight Information Lookup – Fast search using Flight ID.

4. Air Route Connectivity – Airports connected via routes.

5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead

luggage management simulation.

Student Task:

• Choose suitable data structures from the provided list.

• Provide 2–3 sentence justification per feature.

• Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

• Mapping table.

• Working Python implementation.

Emergency Landing priority

Code:

```
"""
Emergency Landing System using Priority Queue

Aircraft with higher priority (emergency) will be allowed
to land before normal aircraft.
"""

import heapq

class AirportLandingSystem:
    def __init__(self):
        # Priority queue to store aircraft
        self.queue = []

    def request_landing(self, priority, flight_id):
        """Add aircraft landing request"""
        heapq.heappush(self.queue, (priority, flight_id))
        print(f"Flight {flight_id} added to landing queue.")

    def allow_landing(self):
        """Allow highest priority aircraft to land"""
        if self.queue:
            priority, flight = heapq.heappop(self.queue)
            print(f"Flight {flight} is landing (Priority {priority}).")
        else:
            print("No aircraft waiting to land.")


# Example usage
if __name__ == "__main__":
    system = AirportLandingSystem()

    system.request_landing(3, "FL101")  # Normal flight
    system.request_landing(1, "FL999")  # Emergency flight
    system.request_landing(2, "FL202")

    system.allow_landing()
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Loc
Flight FL101 added to landing queue.
Flight FL999 added to landing queue.
Flight FL202 added to landing queue.
Flight FL999 is landing (Priority 1).
Flight FL202 is landing (Priority 2).
```

Justification: Aircraft in emergency situations must land before other flights. A priority queue allows flights with higher priority (emergency) to be handled first. This improves safety and quick response.

Task Description #17: Smart Social Media Platform – Data Structure

Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.

2. Trending Hashtags Tracker – Rank hashtags by usage frequency.

3. User Profile Search – Quick lookup using username.

4. Friend Network Representation – Represent user connections.

5. Message Undo Feature – Allow last sent message to be

withdrawn.

Student Task:

• Select appropriate data structures from the list.

• Justify your choices clearly.

• Implement one selected feature as a Python program using AI-

assisted code generation.

Expected Output:

• Table mapping features to data structures with justification.

• Working Python code with docstrings and comments.

User profile search

Code:

```
"""
User Profile Search using Hash Table

This program stores and searches user profiles using usernames.

"""

class UserProfileSystem:
    def __init__(self):
        # Hash table to store user profiles
        self.users = {}

    def add_user(self, username, name):
        """Add a new user profile"""
        self.users[username] = name
        print(f"User {username} added.")

    def search_user(self, username):
        """Search for a user profile"""
        if username in self.users:
            print(f"User found: {username} -> {self.users[username]}")
        else:
            print("User not found.")


# Example usage
if __name__ == "__main__":
    system = UserProfileSystem()

    system.add_user("hasini", "Hasini ")
    system.add_user("anvitha", "Anvitha Reddy")

    system.search_user("hasini")
    system.search_user("Anvitha")
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppDa
User hasini added.
User anvitha added.
User found: hasini -> Hasini
User not found.
```

Justification: A hash table stores user profiles using a unique username as the key. This allows very fast searching and retrieval of user details. It helps the platform quickly find user profiles.

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment

2. Medical Emergency Handling

3. Student Record Retrieval

4. Hostel Block Connectivity

5. Visitor Entry Log

Student Task

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue

o Priority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature with comments and docstrings.

Room Allotment

Code:

```
"""
Hostel Room Allotment System using Queue
This program manages students waiting for hostel rooms.
Rooms are allotted in the order students apply (FIFO).
"""

class RoomAllotmentQueue:
    def __init__(self):
        # Queue to store students
        self.queue = []

    def apply_for_room(self, student_name):
        """Add student to room allotment queue"""
        self.queue.append(student_name)
        print(f"{student_name} added to room allotment queue.")

    def allot_room(self):
        """Allot room to the first student in queue"""
        if self.queue:
            student = self.queue.pop(0)
            print(f"Room allotted to {student}.")
        else:
            print("No students waiting for room.")

    def display_queue(self):
        """Display students waiting for rooms"""
        print("\nStudents waiting for rooms:")
        for student in self.queue:
            print(student)
```

```
# Example usage
if __name__ == "__main__":
    hostel = RoomAllotmentQueue()

    hostel.apply_for_room("Rahul")
    hostel.apply_for_room("Arjun")
    hostel.apply_for_room("Kiran")

    hostel.display_queue()

    hostel.allot_room()
    hostel.display_queue()
```

Output:

```
PS C:\Users\hasin> & C:/Users/hasin/AppData/Loc
Rahul added to room allotment queue.
Arjun added to room allotment queue.
Kiran added to room allotment queue.

Students waiting for rooms:
Rahul
Arjun
Kiran
Room allotted to Rahul.

Students waiting for rooms:
Arjun
Kiran
```

Justification: A queue follows the FIFO (First In First Out) rule. Students who apply first for hostel rooms should get rooms first. This ensures fair room allocation.

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling

2. Fault Priority Handling

3. Consumer Record Lookup

4. Power Grid Connectivity

5. Meter Reading History Tracking

Student Task

• For each feature, select the most appropriate data structure from

the list below:

o Stack

o QueuePriority Queue

o Linked List

o Binary Search Tree (BST)

o Graph

o Hash Table

o Deque

• Justify your choice in 2–3 sentences per feature.

• Implement one selected feature as a working Python program with

AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.

• A functional Python program implementing the chosen feature

with comments and docstrings.

Complaint Handling

Code:

```
"""
Electricity Complaint Handling System using Queue

This program manages consumer complaints.
Complaints are handled in the order they are received (FIFO).
"""

class ComplaintQueue:
    def __init__(self):
        # Queue to store complaints
        self.queue = []

    def register_complaint(self, complaint_id):
        """Add a complaint to the queue"""
        self.queue.append(complaint_id)
        print(f"Complaint {complaint_id} registered.")

    def resolve_complaint(self):
        """Resolve the first complaint in the queue"""
        if self.queue:
            complaint = self.queue.pop(0)
            print(f"Complaint {complaint} resolved.")
        else:
            print("No complaints to resolve.")

    def display_complaints(self):
        """Display all pending complaints"""
        print("\nPending Complaints:")
        for complaint in self.queue:
            print(complaint)
```

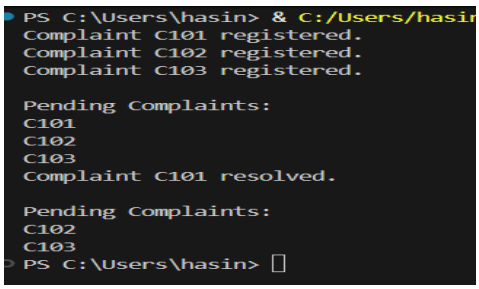
```
# Example usage
if __name__ == "__main__":
    system = ComplaintQueue()

    system.register_complaint("C101")
    system.register_complaint("C102")
    system.register_complaint("C103")

    system.display_complaints()

    system.resolve_complaint()
    system.display_complaints()
```

Output:



Justification: A queue follows the **FIFO (First In First Out)** rule. Complaints that are registered first should be handled first. This ensures fairness in processing customer complaints.